Homework 3:

Optimization and Linear Regression

CPE 486/586

Instructor: Rahul Bhadani

Due: October 11, 2026, 11:59 PM
125 Points

You are allowed to use a generative model-based AI tool for your assignment. However, you must submit an accompanying reflection report detailing how you used the AI tool, the specific query you made, and how it improved your understanding of the subject. You are also required to submit screenshots of your conversation with any large language model (LLM) or equivalent conversational AI, clearly showing the prompts and your login avatar. Some conversational AIs provide a way to share a conversation link, and such a link is desirable for authenticity. Failure to do so may result in actions taken in compliance with the plagiarism policy.

Additionally, you must include your thoughts on how you would approach the assignment if such a tool were not available. Failure to provide a reflection report for every assignment where an AI tool is used may result in a penalty, and subsequent actions will be taken in line with the plagiarism policy.

Submission instruction:

You may either complete your entire homework on Python Notebook and submit a Google Colab link or you may choose a combination of a pdf and Google Colab Notebook. However, you must provide **publicly accessible** Google Colab URL on Canvas.

For .pdf on Canvas, follow the format `CPE486586-LastFirst-HW-XX`. For example, if your name is Sam Wells, your file name should be `CPE486586-WellsSam-HW-XX.pdf`.

Your submission must contain your name and UAH Charger ID or your UAH email address. Please number your pages as well.

Please write down your unique keyword corresponding to your package and your Github account URL. Do not provide a link to your Google Colab notebook. Upload your Google colab notebook to your repo in a new folder called HWNotebooks. Provide your GitHub commit hash. Your submitted notebook must be fully executed

with all cells outputs. Please make sure there are no error in your submitted notebook.

Theory

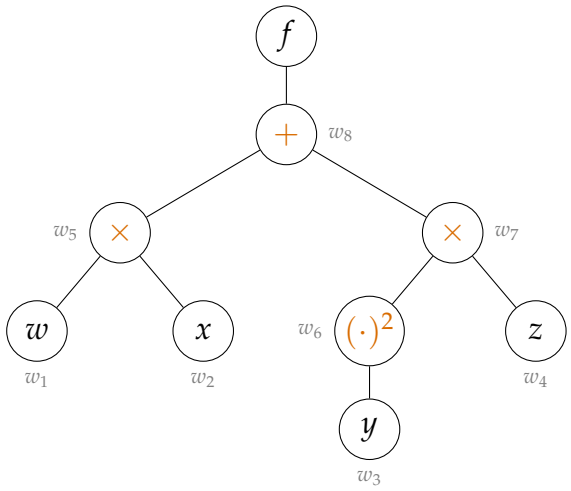
1 Automatic Differentiation and Expression Graph (10 Points)

Consider a function $f(w, x, y, z) = wx + y^2 z$. Draw the expression graph corresponding to the given function and demonstrate how would you evaluate $\frac{\partial f}{\partial w}$ for $w = 1$, $x = 2$, $y = 3$, and $z = 4$.

Answer

Let:

$$
\begin{aligned}
w_1 &= w = 1 & w_2 &= x = 2 \\
w_3 &= y = 3 & w_4 &= z = 4 \\
w_5 &= w_1 w_2 = 2 & w_6 &= w_3^2 = 9 \\
w_7 &= w_6 w_4 = 36 & w_8 &= w_5 + w_7 = 38 \\
f &= w_8 = 38
\end{aligned} \tag{1}
$$



$$
\begin{aligned}
\frac{\partial w_5}{\partial w_1} &= \frac{\partial (w_1 \cdot w_2)}{\partial w_1} = w_2 \\
\frac{\partial w_5}{\partial w_2} &= \frac{\partial (w_1 \cdot w_2)}{\partial w_2} = w_1 \\
\frac{\partial w_6}{\partial w_3} &= \frac{\partial (w_3^2)}{\partial w_3} = 2w_3 \\
\frac{\partial w_7}{\partial w_6} &= \frac{\partial (w_6 \cdot w_4)}{\partial w_6} = w_4
\end{aligned} \tag{2}
$$

$$
\begin{aligned}
\frac{\partial w_7}{\partial w_4} &= \frac{\partial (w_6 \cdot w_4)}{\partial w_4} = w_6 \\
\frac{\partial w_8}{\partial w_5} &= \frac{\partial (w_5 + w_7)}{\partial w_5} = 1 \\
\frac{\partial w_8}{\partial w_7} &= \frac{\partial (w_5 + w_7)}{\partial w_7} = 1 \\
\frac{\partial f}{\partial w_8} &= 1
\end{aligned} \tag{3}
$$

Apply chain rule:

$$\begin{aligned}\frac{\partial f}{\partial w_1} &= \frac{\partial f}{\partial w_8} \times \frac{\partial w_8}{\partial w_5} \times \frac{\partial w_5}{\partial w_1} \\ &= 1 \times 1 \times w_2 \\ &= 1 \times 1 \times 2 = 2\end{aligned} \tag{4}$$

2 Unconstrained Optimization (5 Points)

1. Minimize $f(x) = x^2 - 4x + 4$, determine the critical point, and repor the minimum value of the function. **(5 Points)**
2. Optimize $f(x) = x^3 - 3x^2$. Find all the critical points possible. Which one of them are local minima and local maxima? What is the local minimum and maxmium values of $f(x)$? **(10 Points)**
3. Consider $f(x, y) = x^2 - y^2$. Compute its gradient ∇f and associated critical point. Is this critical point a maximum, a minimum, or a saddle point? **(5 Points)**

Answer

1.

$$\begin{aligned}f(x) &= x^2 - 4x + 4 \\ \frac{\partial f(x)}{\partial x} &= 2x - 4 = 0 \\ x &= 2 \\ \frac{\partial^2 f(x)}{\partial x^2} &= 2 > 0, \quad \text{critical point is a local minimum}\end{aligned}$$

Minimum value is $f(2) = 2^2 - 4 \cdot 2 + 4 = 4 - 8 + 4 = 0$.

2.

$$\frac{\partial f(x)}{\partial x} = 3x^2 - 6x = 0$$
$$3x(x-2) = 0 \Rightarrow x = 2, x = 0$$
$$\frac{\partial^2 f(x)}{\partial x^2} = 6x - 6$$
$$6x - 6\Big|_{x=0} = -6 < 0, \qquad \text{critical point is a local maximum}$$
$$6x - 6\Big|_{x=2} = 6 > 0 \qquad \text{critical point is a local minimum}$$
$$f(0) = 0$$
$$f(2) = 8 - 3 \cdot 4 = -4$$

3.

Given $f(x,y) = x^2 - y^2$

The gradient is

$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right)$$
$$\frac{\partial f}{\partial x} = 2x$$
$$\frac{\partial f}{\partial y} = -2y$$
$$\nabla f = (2x, -2y)$$

Set $\nabla f = (0,0)$ to find critical points:

$$2x = 0 \Rightarrow x = 0$$
$$-2y = 0 \Rightarrow y = 0$$

Critical point: $(0,0)$

Second derivatives:

$$\frac{\partial^2 f}{\partial x^2} = 2$$
$$\frac{\partial^2 f}{\partial y^2} = -2$$
$$\frac{\partial^2 f}{\partial x \partial y} = 0$$

Hessian matrix: $H = \begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix}$

$$\det(H) = (2)(-2) - (0)(0) = -4$$

Since $\det(H) < 0$, the critical point $(0, 0)$ is a saddle point.

3 Constrained Optimization using Lagrange Multiplier (10 Points)

Using method of Lagrange mulitplier, minimize $f(x, y, z) = x^2 + y^2 + z^2$ subject to $x + 2y + z = 6$.

1. Set up the Lagrangian.
2. Derive the system of equations.
3. Solve for critical points.
4. Classify extrema.
5. Explain the role of the constraint.

Answer

Step 1: Set up the Lagrangian

We minimize

$$f(x, y, z) = x^2 + y^2 + z^2$$

subject to the constraint

$$g(x, y, z) = x + 2y + z = 6.$$

The Lagrangian is:

$$\mathcal{L}(x, y, z, \lambda) = x^2 + y^2 + z^2 - \lambda(x + 2y + z - 6)$$

Step 2: Derive the system of equations

$$\frac{\partial \mathcal{L}}{\partial x} = 2x - \lambda = 0$$
$$\frac{\partial \mathcal{L}}{\partial y} = 2y - 2\lambda = 0$$
$$\frac{\partial \mathcal{L}}{\partial z} = 2z - \lambda = 0$$

Step 3: Solve for critical points

From (1): $x = \frac{\lambda}{2}$
From (2): $y = \lambda$
From (3): $z = \frac{\lambda}{2}$
Substitute into (4):

$$\frac{\lambda}{2} + 2\lambda + \frac{\lambda}{2} = 6 \Rightarrow 3\lambda = 6 \Rightarrow \lambda = 2$$

Thus:

$$x = 1, \quad y = 2, \quad z = 1$$

Critical point: $(1, 2, 1)$

Step 4: Classify extrema

The function $f(x, y, z) = x^2 + y^2 + z^2$ is convex (positive definite quadratic form) and the constraint is linear. Therefore, the critical point $(1, 2, 1)$ is a **minimum**.
Minimum value:

$$f(1, 2, 1) = 1^2 + 2^2 + 1^2 = 6$$

Step 5: Role of the constraint

The constraint $x + 2y + z = 6$ defines a plane in $\mathbb{R}^3$. Minimizing $f(x, y, z)$ subject to this constraint finds the point on the plane closest to the origin (since f represents squared distance from origin). The Lagrange multiplier method gives the projection of the origin onto this plane.

4 Optimizing a Function using Stochastic Gradient Descent (5 Points)

Use the Stochastic Gradient Descent technique to calculate the optimal value of w_1 for the cost function shown in Equation (1). Use a learning rate $\eta = 0.1$, and an initial value $w_1^{(0)} = 2.0$. Write down the updated value of w_1 for 5 iterations using the formula for gradient descent.
This question is **analytical**, no code is required.

$$J(w_1) = (w_1 - 5)^4 + (w_1 - 3)^2 + 4 \tag{5}$$

How does it compare with optimizing J using partial differentiation method?

Answer

The cost function is

$$J(w_1) = (w_1 - 5)^4 + (w_1 - 3)^2 + 4$$

with gradient

$$\frac{\partial J}{\partial w_1} = 4(w_1 - 5)^3 + 2(w_1 - 3).$$

Using stochastic gradient descent (SGD) with learning rate $\eta = 0.1$ and initial value $w_1^{(0)} = 2.0$, the update rule is:

$$w_1^{(t+1)} = w_1^{(t)} - \eta \cdot \frac{\partial J}{\partial w_1} = w_1^{(t)} - 0.4\,(w_1^{(t)} - 5)^3 - 0.2\,(w_1^{(t)} - 3).$$

Iterations:

$$w_1^{(0)} = 2.0$$

$$\left.\frac{\partial J}{\partial w_1}\right|_{2.0} = 4(-3)^3 + 2(-1) = -110 \quad \Rightarrow \; w_1^{(1)} = 2.0 - 0.1(-110) = 13.0$$

$$\left.\frac{\partial J}{\partial w_1}\right|_{13.0} = 4(8)^3 + 2(10) = 2068 \quad \Rightarrow \; w_1^{(2)} = 13.0 - 0.1(2068) = -193.8$$

$$\left.\frac{\partial J}{\partial w_1}\right|_{-193.8} = 4(-198.8)^3 + 2(-196.8) \approx -3.1428 \times 10^7 \quad \Rightarrow \; w_1^{(3)} \approx 3.1426 \times 10^6$$

$$\left.\frac{\partial J}{\partial w_1}\right|_{w_1^{(3)}} \approx 4(3.1426 \times 10^6)^3 \approx 1.2414 \times 10^{20} \quad \Rightarrow \; w_1^{(4)} \approx -1.2414 \times 10^{19}$$

$$\left.\frac{\partial J}{\partial w_1}\right|_{w_1^{(4)}} \approx 4(-1.2414 \times 10^{19})^3 \approx -7.653 \times 10^{57} \quad \Rightarrow \; w_1^{(5)} \approx 7.653 \times 10^{56}$$

The iterates oscillate in sign and grow without bound, so SGD **diverges** for $\eta = 0.1$. The cause is the curvature

$$\frac{\partial^2 J}{\partial w_1^2} = 12(w_1 - 5)^2 + 2,$$

which equals 110 at the starting point. A gradient step is stable only when $\eta < 2/J''(w_1)$, which here requires $\eta < 2/110 \approx 0.018$. The first step overshoots to $w_1 = 13$, where the curvature is larger still ($J'' = 770$), and each later overshoot is worse than the one before. A smaller learning rate such as $\eta = 0.01$ would converge, with the first step landing at $w_1^{(1)} = 3.1$.

Comparison with Partial Differentiation:
Setting the gradient to zero and substituting $u = w_1 - 5$:

$$4u^3 + 2(u + 2) = 0 \implies 2u^3 + u + 2 = 0.$$

The cubic has derivative $6u^2 + 1 > 0$, so it has exactly one real root. Cardano's formula gives

$$u = \sqrt[3]{-\tfrac{1}{2} + \sqrt{\tfrac{55}{216}}} + \sqrt[3]{-\tfrac{1}{2} - \sqrt{\tfrac{55}{216}}} \approx -0.8351,$$

so

$$w_1^* \approx 4.1649, \qquad J(w_1^*) \approx 5.8433.$$

Since $J''(w_1) > 0$ for all w_1, J is strictly convex and this stationary point is the unique global minimum.

Partial differentiation finds the optimum directly, though it requires solving a cubic, which has no simple closed form in practice. SGD avoids solving the equation, but its success depends on the learning rate. Convexity guarantees a single minimum, yet it does not guarantee convergence: with $\eta = 0.1$ the method moves away from $w_1^* \approx 4.1649$ instead of towards it. In the quadratic example the curvature was constant, so one fixed η worked everywhere. The quartic term here makes the safe step size depend on where the iterates are.

5 Linear Regression and Regularization (15 points)

In class we derived and discussed linear regression in detail. Find the result of minimizing the loss of sum of the squared errors after adding in a penalty called an L_2 penalty on the weights. More formally

$$\arg\min_{\mathbf{w}} \left\{ ||\mathbf{y} - \mathbf{x}\mathbf{w}||^2 + \lambda ||\mathbf{w}||^2 \right\} \tag{6}$$

which means we are interested in finding $\mathbf{w}$ that minimizes the overall expression. How does this change the solution to the original linear regression solution? What is the impact of adding in this penalty?

$||\cdot||$ is L2 norm.

Hint: First take the gradient with respect to $\mathbf{w}$ and then equate to zero you will obtain an expression for $\mathbf{w}$ which is the estimated $\hat{\mathbf{w}}$.

Also, remember in vector form the L2 norm $||v||$ can be written as $\mathbf{v}^\top \mathbf{v}$.

Answer

$$\arg\min_{\mathbf{w}} \left\{ ||\mathbf{y} - \mathbf{x}\mathbf{w}||^2 + \lambda ||\mathbf{w}||^2 \right\}$$

$$\frac{d}{d\mathbf{w}} \left\{ \frac{1}{2}(\mathbf{y} - \mathbf{x}\mathbf{w})^\top (\mathbf{y} - \mathbf{x}\mathbf{w}) + \frac{\lambda}{2}\mathbf{w}^\top \mathbf{w} \right\} = -\mathbf{x}^\top (\mathbf{y} - \mathbf{x}\mathbf{w}) + \lambda \mathbf{w} = 0$$

$$\begin{aligned} -\mathbf{x}^\top (\mathbf{y} - \mathbf{x}\mathbf{w}) + \lambda \mathbf{I}\mathbf{w} &= 0 \\ (\mathbf{x}^\top \mathbf{x} + \lambda \mathbf{I})\mathbf{w} &= \mathbf{x}^\top \mathbf{y} \\ \mathbf{w} &= (\mathbf{x}^\top \mathbf{x} + \lambda \mathbf{I})^{-1} \mathbf{x}^\top \mathbf{y} \end{aligned}$$

The original solution (without regularization) is $\mathbf{w} = (\mathbf{x}^\top \mathbf{x})^{-1} \mathbf{x}^\top \mathbf{y}$. Compared to this, with regularization, an extra term is $\lambda \mathbf{I}$. The impact of adding penalty

is that it penalizes any data points far from the regressed line, so that the slope of the line is not affected too much, and tends to be less steep. This results in model preventing from overfitting.

6 Assumptions in Linear Regression (5 Points)

Write all the assumptions while considering simple linear regression model.

Answer

For the simple linear regression model:

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

where Y is the dependent variable, X is the independent variable, β_0 and β_1 are parameters, and ε is the error term.

1. **Linearity**: The relationship between X and Y is linear:
$$E(Y|X) = \beta_0 + \beta_1 X$$
2. **Independence**: Observations (X_i, Y_i) are independently distributed.
3. **Homoscedasticity**: Constant variance of errors:
$$\text{Var}(\varepsilon_i|X_i) = \sigma^2 \quad \text{for all } i$$
4. **Normality**: The error terms are normally distributed:
$$\varepsilon_i \sim N(0, \sigma^2)$$
5. **No perfect multicollinearity**: In simple linear regression, this means X is not constant (there is variation in X values).
6. **Exogeneity**: The error term has zero conditional mean:
$$E(\varepsilon_i|X_i) = 0$$
7. **Random sampling**: The sample is randomly drawn from the population.
8. **No autocorrelation**: Error terms are uncorrelated with each other:
$$\text{Cov}(\varepsilon_i, \varepsilon_j) = 0 \quad \text{for } i \neq j$$
9. **No measurement error**: X is measured without error (or with negligible error).

Practice

Note that you don't need to use Google Colab for submission.

7 Computing Derivative using PyTorch Autograd (5 Points)

Consider a following function

$$\tau = 2r \cdot \sqrt{\frac{r}{r_s}} \cdot \sqrt{1 - \frac{r_s}{r}} \tag{7}$$

which is the time experienced by an observer traveling along a specific trajectory in curved spacetime. r is the radial coordinate in a spherical coordinate system and r_s is the Schwarzschild radius.

Write a program in PyTorch to compute the derivative $\frac{d\tau}{dr}$ considering the value of $r = 30000.1$ for $r_s = 0.00887$.

8 Linear Regression in One Variable with Scikit-Learn (35 Points)

Consider the dataset https://github.com/rahulbhadani/Datasets/blob/master/RAV4/2020-04-15-05-12-00_2T3Y1RFV8KC014025_CAN_Messages_5_Hz.csv that contains various driving features from a test driving.

In your Google Colab notebook, use Scikit-learn to create a linear regression model with steer angle as independent variable and yaw rate as response variable. You need to split the dataset into training and test with 25% of the rows set aside as a test data. Report the estimated coefficients of the fitted linear model and R^2 computed using the test data.

9 Linear Regression in One Variable with PyTorch (35 Points)

Consider the dataset https://github.com/rahulbhadani/Datasets/blob/master/Hydropower.csv. The main task as a part of this assignment is to create a linear model $y = w_0 + w_1 x$ and determine the suitable parameter w_0 and w_1. For this consider the independent variable x as Benefit-Cost-Ratio (BCR) from the given dataset and the response variable y as AnnualProduction. Using the linear model, we should be able to predict Annual Production given a new BCR.

You need to write LinearRegression class based on gradient descent implemented using PyTorch as a part of your Python package with the following specification:

1. Create a subpackage **model** in your UV Python package. **(2 Points)**
2. The subpackage should contain a module `regression.py`. **(2 Points)**
3. Add a `LinearRegression` class to the module with the following requirement:
   (a) `__init__` function, which is a constructor should take following parameters as argument: (i) learning rate, and (ii) number of epochs. Further, `__init__` should initialize w_0 and w_1, SGD optimizer, and mean-squared error as a loss function, and list or vector to store intermediate w_0, w_1, and loss as the training progresses. **(5 Points)**
   (b) `forward` function should specify the linear model and compute the response y and return it. **(2 Points)**
   (c) `fit` function should perform machine learning on training dataset provided to the function as an argument. fit function also should take test data as an additional argument and after training, it should compute R^2 on the test data and print it. **(5 Points)**
   (d) `predict` function should predict AnnualProduction given a new BCR data. **(4 Points)**
   (e) `analysis_plot` function should create several plots on the same figure using subplots such as original data, fitted regression line, nd w_0 , w_1 and loss as the training progressed. **(5 Points)**
4. Add appropriate entry to `__init__.py` at subpackage level and at package level to facilitate proper import. **(2 Points)**
5. Build your package after incrementing version number, commit to GitHub, and publish to PYPI. **(3 Points)**
6. Show the complete usage of your LinearRegression model with `Hydropower.csv` dataset through a standalone notebook that first installs your package. **(5 Points)**